

Response-Level Rewards Are Equivalent to Token-Level Rewards: Mathematical Principles for Online Reinforcement Learning

May 5, 2026

1 Summary

Starting from first-principles derivations, this work addresses several common questions in online RL: what impact does the **zero-reward assumption for intermediate tokens** have on training algorithms? What is the essential difference between algorithms built on this assumption and systems trained with explicit **token-level rewards**? What are the differences in **theoretical modeling capacity** between REINFORCE-style methods such as GRPO, RLOO, and ReMax, and PPO? What role does reward discounting γ play?

RL formulas often look complicated, but LLM RL has special structure that makes relatively precise analysis easier.

2 Zero-Reward Assumption for Intermediate Tokens

This assumption is very common in typical scenarios. In question answering, reward models usually score only the overall quality of a complete response. In mathematical reasoning, reward can be simplified to whether the final answer is correct.

Manually annotating intermediate steps and training token-level reward models is labor-intensive and can be unreliable. Therefore, in practice, people often assume intermediate generated tokens receive no direct reward, and only the last token receives the score for the whole response. This is the **zero-reward assumption for intermediate tokens**.

At first glance, this assumption seems unreasonable. For humans, the meaning of a sentence is often clear even before seeing the final token. We may not be able to quantify each token’s exact information contribution, but those contributions do exist; they are just hard to compute.

Several representative works over the past two years discussed this concern. ReMax argued RLHF training is closer to a **single-stage** rather than **multi-stage** process [1]. GRPO argued that with zero intermediate rewards, computing value functions accurately at **every position** is complex and unnecessary [2]. RLOO argued that humans care about overall reward and that intermediate “true” rewards do not really exist [7]. Based on this, these works suggest PPO’s value network is unnecessary.

Naturally, one asks: if response-level reward can be decomposed across tokens, how different is RL trained with the **zero-reward assumption for intermediate tokens** from RL trained with exact token-level rewards?

A representative approach is the ABC model [8], which uses the final-layer attention of the reward model to decompose overall reward and reports gains. Their experiments use non-logic reasoning tasks where response-level reward is a scalar. What about logic reasoning tasks where response-level reward is only 0/1?

Our work derives the theoretical difference.

3 Fundamental Principles

Nearly all LLM RL work starts from the classic Policy Gradient Theorem. This derivation is general, and usually implicitly assumes reward discount factor $\gamma = 1$; otherwise the form becomes more complex rather than the compact expression commonly shown.

LLM RL has a special property: RL state corresponds to generated token sequences. We use $\mathbf{W}$ for a full trajectory, w_0 for the prompt, and $\mathbf{W}_{i,j}$ for a substring. Then any two states are either unreachable or connected by exactly one path. This allows us to introduce discounting precisely and obtain an equivalent form.

Lemma 1. *In LLMs application scenarios, the policy gradient theorem with a discounted reward factor γ would be*

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T \gamma^{t-1} Q_{\pi_{\theta}}(\mathbf{W}_{0,t-1}, w_t) \nabla \log \pi_{\theta}(w_t | \mathbf{W}_{0,t-1}) \quad (4)$$

where $\mathbf{W} = w_0 w_1 w_2 \dots w_T \sim \pi_{\theta}(\mathbf{W} | w_0)$.

Compared with the classic Gradient Policy Theorem [9], this form differs in two ways. First, the classic result is proportional to the right-hand side, while in LLM RL it is an equality, enabling exact analysis of how to choose γ . Second, in the classic form the sampling distribution differs from the policy-gradient target distribution π_{θ} , creating a difficult bias problem [10], while in LLM RL they are equal.

Recall that $Q_{\pi_{\theta}}$ is also an expected future return. It and the leftmost $\mathbb{E}_{\mathbf{W}}$ in Lemma 1 are separate expectations but under the same distribution. In practice, we directly use sampled $\mathbf{W}$ to approximate $Q_{\pi_{\theta}}$, redefining it as G_t , the realized reward from position t to trajectory end. Although using one sample has high variance, it is unbiased.

Under the **zero-reward assumption for intermediate tokens**, each position's G_t equals whole-response reward $RM(\mathbf{W})$, yielding the core formula used by GRPO, ReMax, and RLOO:

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T \gamma^{t-1} RM(\mathbf{W}) \nabla \log \pi_{\theta}(w_t | \mathbf{W}_{0,t-1})$$

The next key theorem was originally used to reduce policy-gradient variance; here we also use it to prove our core result.

Lemma 2. Adding a state-dependent constant $c(\mathbf{W}_{0,t-1})$ to Lemma 1 does not affect the expectation of the policy gradient.

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T (c(\mathbf{W}_{0,t-1}) + \gamma^{t-1} Q_{\pi_\theta}(\mathbf{W}_{0,t-1}, w_t)) \nabla \log \pi_\theta(w_t | \mathbf{W}_{0,t-1})$$

where $\mathbf{W} = w_0 w_1 w_2 \dots w_T \sim \pi_\theta(\mathbf{W} | w_0)$.

It shows that adding a state-dependent constant c on $\mathbf{W}_{0,t-1}$ does not change the policy gradient. So one typically sets c as the value function $V_{\pi_\theta}(\mathbf{W}_{0,t-1})$, which can be shown to be **near-optimal**.

Now consider setting $c(\mathbf{W}_{0,t-1}) = \sum_{k=1}^{t-1} \gamma^{k-1} r(w_k)$, where only γ is variable and each $r(w_k)$ is objectively present but hard to compute exactly. Then we naturally obtain:

Theorem 1. In LLM application scenarios, when the response-level reward is defined in such a form $RM(\mathbf{W} = w_1 w_2 \dots w_T) = \sum_{t=1}^T \gamma^{t-1} r(\mathbf{W}_{0,t-1}, w_t)$ for real token rewards $r(\mathbf{W}_{0,t-1}, w_t)$ and some discounted reward factor γ , then the policy gradient can be unbiasedly estimated from mere response-level rewards, regardless of the values of $r(\mathbf{W}_{0,t-1}, w_t)$ and γ .

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T \mathbb{E}_{\mathbf{W}^{(t+1)}} RM(\mathbf{W}^{(t+1)}) \nabla \log \pi_\theta(w_t | \mathbf{W}_{0,t-1}) \quad (21)$$

where $\mathbf{W} \sim \pi_\theta(\mathbf{W} | w_0)$; $\mathbf{W}^{(t)} \sim \pi_\theta(\mathbf{W} | w_0)$ is a full trajectory, s.t. $\mathbf{W}_{0,t-1}^{(t)} = \mathbf{W}_{0,t-1}$; the $RM(\mathbf{W})$ can be defined on task-specific outcome or reasoning steps, or both.

where $\mathbf{W}^{(t+1)}$ is a complete random trajectory whose first t tokens match those of current trajectory $\mathbf{W}$.

This formula implies two things. First, exact policy gradients do not require direct rewards at each intermediate position; complete-trajectory reward is sufficient. This is analogous to **sufficient statistics**. Since current sampled $\mathbf{W}$ is also one valid $\mathbf{W}^{(t+1)}$, we can approximate with current $\mathbf{W}$, recovering GRPO/ReMax/RLOO formulas. This means for REINFORCE-style methods, policy gradients derived under the **zero-reward assumption for intermediate tokens** are equivalent to those derived from true-but-unknown token-level rewards.

Second, discount factor γ disappears from the policy gradient via cancellation in derivation. Under the common setup where final-token reward is not discounted, this implicitly means $\gamma = 1$. What if we keep that setup but set $\gamma \neq 1$? We then get:

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T \frac{1}{\gamma^{t-1}} \mathbb{E}_{\mathbf{W}^{(t+1)}} RM(\mathbf{W}^{(t+1)}) \nabla \log \pi_\theta(w_t | \mathbf{W}_{0,t-1}^{(t)}) \quad (26)$$

A contradiction appears: if future rewards are discounted with $\gamma < 1$, later positions receive larger weights $(\frac{1}{\gamma})^{t-1}$. Therefore, policy gradient is independent of discounting here, and with the **zero-reward assumption for intermediate tokens**, one should set $\gamma = 1$.

Following PPO, we can also compute a baseline to reduce variance; a **practical approximate solution** is:

Theorem 2. *Theorem 1 with a near-optimal baseline for the gradient variance reduction is*

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T \left(\mathbb{E}_{\mathbf{W}^{(t+1)}} RM(\mathbf{W}^{(t+1)}) - \mathbb{E}_{\mathbf{W}^{(t)}} RM(\mathbf{W}^{(t)}) \right) \nabla \log \pi_{\theta}(w_t | \mathbf{W}_{0,t-1}) \quad (28)$$

Its meaning is clear: for action w_t at time/state t , compare expected returns of trajectories expanded from new state $\mathbf{W}_{0,t}$ vs. from current state $\mathbf{W}_{0,t-1}$. Baseline has an optimal closed-form solution but is inconvenient in practice, so RL commonly uses tractable approximations.

Theorem 2 derives a variance-reduced policy gradient from general token-level rewards. Under the **zero-reward assumption for intermediate tokens**, it naturally becomes the familiar PPO form:

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\mathbf{W}} \sum_{t=1}^T (Q_{\pi_{\theta}}(\mathbf{W}_{0,t-1}, w_t) - V_{\pi_{\theta}}(\mathbf{W}_{0,t-1})) \nabla \log \pi_{\theta}(w_t | \mathbf{W}_{0,t-1})$$

Therefore, **PPO is a general model with token-level reward modeling capability; more precisely, token-level RL and its corresponding response-level RL map to the same gradient update formula**. The intuitively suspicious **zero-reward assumption for intermediate tokens** is theoretically sound.

Also, **REINFORCE-style methods are likewise general token-level reward modelers**. The token-level modeling capability people sought has been inside commonly used methods all along.

Do these models still differ in modeling ability? Yes.

4 Comparison with GRPO, ReMax, RLOO, DPO

Theorem 2 makes two major differences clear, which can affect convergence and capability.

First, each time-step weight contains an expectation term representing action weight. PPO estimates this expectation; although imperfect, it should be more accurate than GRPO/ReMax/RLOO using only one Monte Carlo sample, thus potentially giving faster convergence and higher performance ceilings.

Second, baselines used by GRPO/ReMax/RLOO differ in details but are essentially equivalent to $\mathbb{E}_{\mathbf{W}^{(t=1)}} RM(\mathbf{W}^{(t=1)})$ in Theorem 2. In principle each time t should have its own baseline, but these methods use a global baseline from $t = 1$.

They also use gradient standardization $z = \frac{X - \mu}{\sigma}$. Centering by subtracting mean does not reduce variance; dividing by standard deviation does scale variance, but not in the statistical sense used for unbiased variance reduction. It is more like uniformly shrinking signal amplitudes across positions, somewhat equivalent to a smaller learning rate.

Perplexity’s practical GRPO study on mathematical reasoning [5] observed that when all rewards in a batch are zero, update weights at every time step are also zero under that formula. Under PPO-style estimation, however, some trajectories may be wrong only at final steps, while earlier terms in Formula 2 need not cancel, so useful gradients can still be received at many positions.

For DPO: ignoring that native DPO is offline RL (typically weaker than online RL), and focusing only on objective modeling capacity, original DPO is effectively equivalent to GRPO/ReMax/RLOO. So its theoretic-

cal modeling level is similar. Considering offline characteristics as well, DPO should generally underperform PPO across broad tasks; see experiments in [6].

Overall: on general tasks, PPO’s modeling ability is not worse than popular GRPO/ReMax/RLOO/DPO, provided the critic can be trained effectively.

5 Results on Mathematical Reasoning

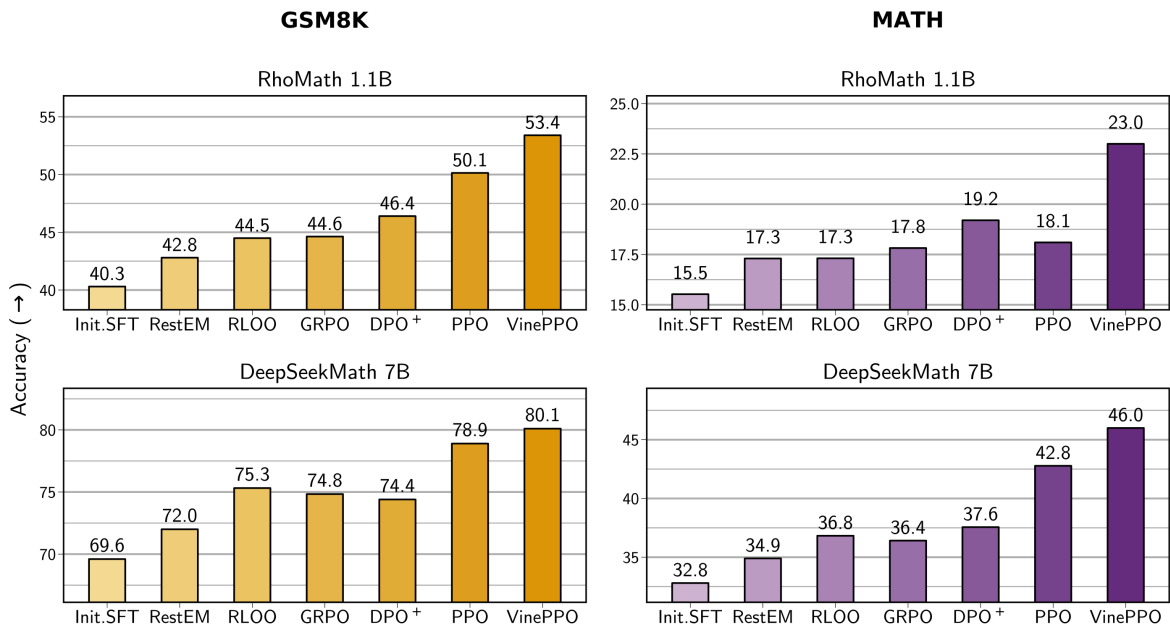
Based on Theorem 2, we propose TRePO, which uses Monte Carlo sampling to estimate the two terms at each time step.

Because sampling is expensive, we do not compute every t . Instead, we randomly select several time steps as an external parameter and compute their advantages accurately; values between these selected steps are approximated by interpolation.

This paper focuses on theory, so we only propose the algorithm without comprehensive experiments. Interestingly, an ICML 2025 paper, VinePPO [3], starting from the **zero-reward assumption for intermediate tokens** and aiming to reduce PPO critic instability, proposes the same sampling-based algorithm.

Both TRePO and VinePPO use Monte Carlo sampling and introduce additional engineering complexity. But in model-complexity terms, they simplify overall training: jointly training policy and critic is usually harder than training policy alone, while Monte Carlo sampling is largely an engineering problem. It also parallelizes naturally and can be accelerated significantly with enough infrastructure, though this is easier for large companies than for smaller teams or schools.

Below is VinePPO’s mathematical-reasoning result figure across multiple models, which also supports conclusions in this paper, e.g., PPO modeling ability is not worse than other popular methods such as GRPO.



6 Implications

This paper mainly analyzes the theoretical effect of the **zero-reward assumption for intermediate tokens** on RL training and removes major concerns about the assumption.

In practice, this suggests we can confidently use it for non-decomposable response-level rewards, such as task-specific and 3H-style traits (helpfulness, honesty, harmlessness).

Token-level reward is a special case of response-level reward and can also be handled with this assumption. Exact per-position Q estimation may reduce practical error, but the two are theoretically equivalent.

When mixing multiple response-level and token-level rewards, one can even uniformly apply the zero assumption and attach reward to the final token.

Comparing major RL algorithms by modeling essence, they roughly reduce to two classes: REINFORCE-style (e.g., GRPO), which performs well and saves memory; actor-critic style (e.g., PPO), which can perform better but at higher cost. Within each class, there may be more algorithmic tricks, but without injecting additional information into the system (e.g., stronger reward models), differences may be limited.

In practice, RL system quality also depends heavily on data and reward model design (e.g., whether to reward/penalize specific reasoning steps), often even more decisively than algorithm choice. If these representative RL algorithms are packaged as black boxes, practitioners can avoid deep RL details, lowering adoption cost and allowing more effort on better reward models, which may improve system performance and accelerate validation of new applications.

7 References

- [1] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. arXiv preprint arXiv:2310.10505, 2023.
- [2] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- [3] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment. arXiv preprint arXiv:2410.01679, 2024.
- [4] AI Team. RL Training For Math Reasoning Introduction and Motivation. <https://www.perplexity.ai/hub/blog/rl-training-for-math-reasoning>. 2025.
- [5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [6] Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. Is dpo superior to ppo for llm alignment? a comprehensive study. arXiv preprint arXiv:2404.10719, 2024.

- [7] Arash Ahmadian, Chris Cremer, Matthias Galle, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet ' Ust " un, " and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740, 2024.
- [8] Alex J Chan, Hao Sun, Samuel Holt, and Mihaela Van Der Schaar. Dense reward for free in reinforcement learning from human feedback. arXiv preprint arXiv:2402.00782, 2024.
- [9] Richard S Sutton and Andrew G Barto. Reinforcement learning: An introduction. second, 2018.
- [10] Weizhen Wang, Jianping He, and Xiaoming Duan. Analysis of on-policy policy gradient methods under the distribution mismatch. arXiv preprint arXiv:2503.22244, 2025.